



Önálló laboratórium beszámoló

Távközlési és Mesterséges Intelligencia Tanszék

Név: **Orbán Levente**

orban.levente.laszlo@gmail.com

Neptun-kód: **CZ6NDB**

Szakirány: **Szoftverfejlesztés**

Konzulens: **Németh Gábor**

nemethgab@tmit.bme.hu

Téma címe: Interaktív SKI-kombinátor kalkulus fordító és vizualizáló

Feladat:

A hallgató feladata egy webes eszköz (Combinator Explorer) elkészítése, amely SKI-kifejezések esetén megmutatja a redukciós lépéseket, vizualizálja a redexeket, és felrajzolja a kiértékelési fát. A redukció során többféle kiértékelési stratégiát támogat, valamint az SKI-kifejezéseket lefordítja λ -kalkulusba. Az eszköz hidat képez a formális elmélet és a gyakorlati megvalósítás között: oktatási segédeszköz, amely mélyíti a funkcionális programozás, illetve a fordítóprogramok működésének megértését.

Tanév: 2025/26. tanév, II. félév

1. A laboratóriumi munka környezetének ismertetése, a munka előzményei és kiindulási állapota

1.1. Bevezető

Az elmúlt három évtizedben egyre nagyobb jelentőségre tett szert a funkcionális programozás az imperatív és objektumorientált paradigmákkal szemben. Az figyelhető meg, hogy azok a programozási nyelvek, amelyek tradicionálisan objektumorientált szemléletűek voltak (például C++, Java) napjainkra jelentős számú funkcionális elemmel bővültek (anonim függvények, lista komprehenzió, magasabb rendű függvények, stb.). Továbbá azok a nyelvek, amelyek az elmúlt évtizedben tettek szert népszerűsége (például Rust) már a kezdetektől fogva egy több paradigmát ötvöző nézettel lettek tervezve. Ezeket az új eszközöket viszont – mind a már piacon lévő mérnökök továbbképzése, mind a felsőoktatásban résztvevők oktatása szempontjából – pedagógiaiilag még a mai napig nehéz kielégítően oktatni. A funkcionális paradigma – illetve annak a matematikai alapjai – idegenek lehetnek a mérnöki beállítottságú közönség számára.

A jelen írásos beszámolómban bemutatott szoftver ezt a pedagógiai hiányt próbálja pótolni: egy olyan vizuális, interaktív, webes eszközt ad a pedagógusok kezébe, amellyel a funkcionális programozást egészen a matematikai alapokig visszamenően lehet hatékonyabban oktatni, hozzá pontosabb intuíciót építeni.

A választott megközelítés (vizuális, webes program) nem egyedi. Számtalan más olyan témakörhöz, amely tradicionálisan pedagógiai kihívásokat jelentett (generált gépi kód, vagy reguláris kifejezések elemzése) készült hasonló alapötletekkel rendelkező szoftver (mint például a Compiler Explorer [9], vagy Regex101 [6]).

A jelen beszámolómban bemutatott megközelítés a modern funkcionális nyelvek matematikai alapjait kívánja megcélozni (többek között redukciós láncok és gráfok, kiértékelési sorrendek, λ - és SKI-kalkulus). Ezen alaptudás ismeretében a legtöbb modern funkcionális nyelv (ML-család, Lisp-család) szemantikáinak a működése – legalábbis magas szinten – könnyebben elsajátítható.

1.2. Elméleti összefoglaló

1.2.1. λ -kalkulus

A modern funkcionális programozási nyelvek alapjaikban a (típus nélküli) λ -kalkulusra építenek¹, amely egy általános célú, Turing-teljes programozási nyelv. Az egyik legegyszerűbb Turing-teljes nyelv, hiszen csak három konstrukcióra épít (lambda-kifejezések, lambda-absztrakciók, és applikációk). A λ -kalkulus nyelvtanának formális definíciója az 1. ábrán látható.

$$\begin{aligned}
 \langle \lambda\text{-kifejezés} \rangle & ::= \langle \text{változó} \rangle \\
 & \quad | \langle \lambda\text{-absztrakció} \rangle \\
 & \quad | \langle \text{applikáció} \rangle \\
 \langle \lambda\text{-absztrakció} \rangle & ::= ' \lambda ' \quad \langle \text{változó} \rangle \quad ' . ' \\
 & \quad \langle \lambda\text{-kifejezés} \rangle \\
 \langle \text{applikáció} \rangle & ::= \langle \lambda\text{-kifejezés} \rangle \\
 & \quad \langle \lambda\text{-kifejezés} \rangle
 \end{aligned}$$

1. ábra. A λ -kalkulus szintaktikája

¹Tekinthetőek a λ -kalkulus kibővített változatának (legyen szó típusokról, vagy let-kötésekről, etc.). Példa erre az ML-család (Standard ML, Caml, Haskell), amelyek közelítőleg felfoghatóak úgy, mint a λ -kalkulus polimorf Hindley-Milner típusrendszerrel való kiegészítése.

A λ -absztrakció segítségével tudunk függvényeket definiálni, amelyeknek paramétereiket az applikáció konstrukciójával tudunk adni. Ha kicsit módosítjuk a nyelvtanunkat, hogy számkonstansokat és egyéb kényelmes konstrukciókat megengedjen, akkor írhatjuk például:

$$\begin{aligned} &\lambda x.(5 * x) \\ &(\lambda x.(5 * x))(3) \\ &15 \end{aligned}$$

Az első sorban definiálunk egy függvényt, amely ötszörözi a bemenetét. A második sorban meghívjuk ezt a függvényt a 3 bemenettel. Az utolsó sorban a szorzás eredménye látható. Fontos megjegyezni, hogy az applikáció bal-asszociatív [13].

1.2.2. Az SKI-kalkulus

A kombinátorok absztrakt matematikai konstrukciók, viszont megértésük és történelmi hátterük kritikus fontosságú a modern számítástudomány, különösen a funkcionális programozás, vagy a kiszámíthatóság modelljeinek vizsgálatához.

Általánosságban különböző problémák formális, szimbolikus ábrázolását már az ókorban megkísérelték. Így született Arisztotelész logikája és Euklidész geometriája. A számítástudomány szempontjából viszont a 19. században a Boole-algebra megjelenése jelentette a legnagyobb áttörést. A 20. század fordulóján a matematikusok arra tettek kísérletet, hogy a teljes matematikát egy szigorú, tiszta logikai alapra helyezték. Ahogy ezek a logikai keretrendszerek egyre nagyobbak lettek, megjelent az igény a bizonyos axiómák meghatározására. Ennek a törekvésnek a legismertebb példája a Sheffer-vonás felfedezése, amely bebizonyította, hogy a teljes ítéletkalkulus felépíthető egyetlen logikai kapu segítségével (NAND) [17]. Felmerült a kérdés: ha a logikai axiómák ilyen elemi szintre redukálhatóak, megtehető-e ugyanez általánosságban a függvényekkel és a változókkal is [13]?

Moses Schönfinkel 1920-ban pontosan erre tett kísérletet [16]. Felismerte, hogy ha a függvényeket kellőképpen általánosítjuk (egy függvény egy másik függvényt is kaphat paraméterként, és visszatérési értéke is lehet egy függvény), a változók feleslegessé válnak. Ezzel az elképzeléssel lefektette a kombinátorkalkulus alapjait. A kombinátorkalkulus leghíresebb és leginkább letisztult² formalizációja az SKI-kalkulus, amelyet Schönfinkel munkássága nyomán Haskell Curry dolgozott ki [4]. Ez pusztán három előre definiált kombinátor egymás utáni alkalmazásával is képes Turing-teljes számítások leírására.

A rendszer a következő három kombinátorból áll:

- Az egyváltozós identitás kombinátor (I), amely beavatkozás nélkül, egyszerűen visszaadja a kapott argumentumát.
- A kétváltozós konstans kombinátor (K), amely olyan függvényt állít elő, ami a második argumentumát mindig figyelmen kívül hagyja, és visszaadja az első.
- A háromváltozós szubsztitúciós kombinátor (S), amely az argumentumok szétosztását és a függvényalkalmazás láncolatát végzi.

Bár a rendszert általában további kombinátorokkal is ki szokás egészíteni a kényelmesség és átláthatóság érdekében, ezek mind levezethetők a fenti három kombinátorból.

Ha a λ -kalkulus és az SKI-kalkulus is Turing-teljesek, akkor bizonyára kell lennie valamilyen ekvivalenciának közöttük. Alább megadom az SKI-kalkulus három kombinátorának a λ -kalkulus szerinti definícióját:

²Bár a fent említett SKI rendszer az általánosan vizsgált forma, fontos megemlíteni, hogy léteznek kevesebb kombinátort igénybe vevő, Turing-teljes rendszerek is. Ilyen például az S és K kombinátorokból álló bázis. Ez abból adódik, hogy az I kombinátor kifejezhető a másik kettőből, például (SKK) módon.

$$\begin{aligned}
 I &= \lambda x.x \\
 K &= \lambda x.(\lambda y.x) \\
 S &= \lambda x.(\lambda y.(\lambda z.xz(yz)))
 \end{aligned}$$

Ezeknek a szabályoknak az alkalmazásával tetszőleges SKI-kalkulus kifejezést át lehet alakítani λ -kalkulus kifejezéssé, melyre a programban lehetőség is van.

1.3. A munka állapota, készütségi foka a félév elején

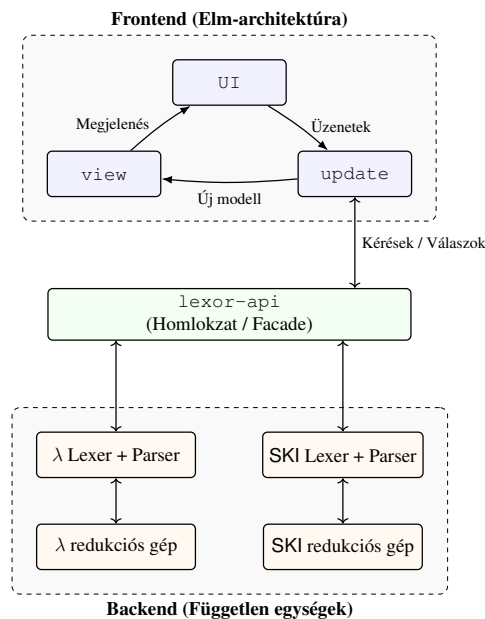
A jelen félév előtti szünetben már elkezdtem feldolgozni a témához releváns irodalmat. A 2025/26/I. félévben egy olyan tárgyat hallgattam, amelyben jelentős szerepe volt a λ -kalkulusnak, kategóriaelméletnek. Ezek az előadások nagyban hozzájárultak az előzetes tudás megszerzéséhez. A konzulensem számos releváns könyvet ajánlott feldolgozásra, többek között Smullyan [18] és Wolfram [19, 20] könyveit és cikkeit.

2. Az elvégzett munka és az eredmények ismertetése

2.1. Az architektúra megtervezése

A program megtervezése során fontos cél volt annak moduláris felépítése és úgynevezett *Separation of Concerns* elve [14]. Ennek megfelelően az elkészült program két jól elkülönülő részre bomlik: frontendre és backendre. Míg a frontend felelős a felhasználóval való kommunikációért, illetve az eredmények megjelenítéséért, addig a backend végzi a tényleges számítási feladatot.

A frontend és a backend közötti tiszta kommunikáció megteremtésére a homlokzat (facade) mintát [8] alkalmaztam. Ez az API réteg (application programming interface) elrejtja a backend komplexitását, és egy könnyen használható interfészt biztosít a frontend számára (az architektúra egyszerűsített vizuális modellje a 2. ábrán látható).



2. ábra. Az alkalmazás rétegei és architektúrája

A backend megtervezése során a fordítóprogramokból merítettem inspirációt [2]. Így a backend is további két részből áll, ún. compiler frontendből és compiler backendből, amelyek közötti kommunikáció egy absztrakt adatstruktúrán, a köztes reprezentáción keresztül történik. Ez a struktúra direkt

azzal a megfontolással lett kialakítva, hogy a compiler backend még optimalizációkat fog végezni rajta, a felhasználó nem interaktál vele. Ilyen például a 2.2.2 fejezetben tárgyalt de Bruijn-forma is.

A compiler frontend tartalmazza a lexert és a parsert. A nyelvtanok egyszerűsége miatt ezt a két elemet egy egységbe összevontam. A compiler backend két redukciós gépet tartalmaz: egyet, ami λ -kalkuluson, egyet, ami SKI-kalkuluson operál.

A frontend terén az úgynevezett *Elm-architektúrát* [5] követtem, amely egy üzenetsorra épülő architektúra. Ez a megközelítés három részre bontja a frontendet:

- A kezdeti `model`-re, amely az applikáció állapotát írja le. Ez az állapot nem mutábilis, tehát ahogy a felhasználó interakcióba lép vele, mindig egy teljesen új állapot jön létre.
- A `view` függvényre, amely a modell megjelenését írja le. A felhasználói interakciókat itt alakítom át üzenetekké.
- Az `update` függvényre, amely az üzenetsort kiolvassa végrehajtja a felhasználó összes parancsát a jelenlegi modellen, ezáltal egy új modellt létrehozva.

Ahogy a felhasználó interakcióba lép a felhasználói felülettel, úgy új üzenetek kerülnek az üzenetsorba. Az üzenetsort minden frame végén az `update` függvény kiüríti, és a tartalmát végrehajtja. Ezáltal egy új modell jön létre, amit átad a `view` függvénynek, ami egy új UI-t hoz létre.

2.2. Redukciós gépek

2.2.1. Az SKI redukciós gép

A redukciós gépek implementálásához a Graph Reduction Machine (G-machine) koncepcióját vizsgáltam meg. Optimalizálási célból a faalapú kiértékelés mellett az alkifejezések megosztása mellett döntöttem, amellyel exponenciálisan kevesebb lépésszámot vesznek igénybe az egyes redukciók [11]. Az elterjedtebb "sztringredukció" – azaz amikor a bemenetet szövegmanipulációs módszerekkel alakítjuk át a redukciós szabályoknak megfelelően – nem skálázódik elég jól hosszabb kifejezések kiértékelésére, illetve az előbb említett alkifejezés-megosztás technikáját sem tudom érvényesíteni.

A követelmények elemzése és a belső reprezentáció kialakítása után implementáltam egy G-machine-t mind a λ -kalkulus, mind az SKI-kalkulus esetében. Az ilyen redukciós gépek implementálása során gyakran használt, faalapú AST (Abstract Syntax Tree, absztrakt szintaxis fa) sokszor memóriatöredezettséghez és lassabb futáshoz vezethet. Ennek elkerülése érdekében a Flat-AST (laposított AST) reprezentációs technikát használtam, amelyet egy aréna alapú memória-allokátorral párosítottam [15]. Egy rekurzív, faalapú megközelítés lehet az 1. kódrészletben bemutatott modell. Itt pointereket tárolunk el a többi csúcsra, amely csökkentheti a memória lokalitását, illetve az adat-

```
1 enum NodeRec {
2     S, K, I,
3     App(Box<Self>, Box<Self>),
4     Indirection(Box<Self>),
5 }
```

1. Kódrészlet. Egy rekurzív, mutató alapú gráf struktúra

struktúra méretét is növelheti 64 bites rendszereken. Ehhez képest az aréna allokátoros megközelítés – melyet a 2. kódrészlet mutat be – nem használ mutatókat. Minden csúcsra az indexe alapján lehet hivatkozni, mely alapján az arénából kinyerhető. Így a különböző kifejezések akár teljesen szomszédos memóriarégiókban lehetnek, növelve a cache-kihasználtságot.

A G-machine a kombinátorok redukcióját egy állapotgép segítségével végzi. A kiértékelés során először végighaladunk a kifejezés "gerincén", amely azokból a csúcsokból áll, amit úgy kapunk, ha a

```

1 enum NodeFlat {
2     S, K, I,
3     App(NodeIdx, NodeIdx),
4     Indirection(NodeIdx),
5 }

```

2. Kódrészlet. Egy aréna allokátoros gráf struktúra

fa gyökerétől kezdve addig megyünk balra, ameddig van baloldali gyerek. A gerincen összegyűlt csúcsok argumentumait a redukciós gép a megfelelő kombinátor szabályai szerint helyettesíti be (például az S, K, I kombinátorok esetében).

Mivel a szoftver célja az oktatás megkönnyítése, több redukciós stratégiát is támogatnom kellett (például normal order, call-by-value). Ezek megértéséhez elemezzük a 3. ábrát [19], amely az alábbi kombinátor AST-ét ábrázolja:

$$S(S(S)(S)(S(S)(K(K)(S))(S)))(S)(S)(S(K(S)(K))(K)(S))$$

Az ábrán látható fában a szürke belső csomópontok az applikációkat (függvényalkalmazásokat) jelölik, a levelek pedig az egyes kombinátorokat. Látható, hogy egy ilyen komplex kifejezés esetén egyszerre több ponton is elvégezhető lenne a redukció. A támogatott stratégiák pontosan abban térnek el, hogy ezek közül melyiket választják:

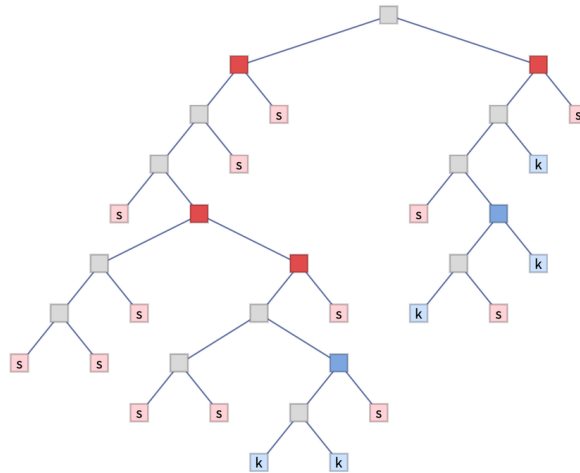
- Normál sorrend (normal order és call-by-name): Mindig a legbaloldalibb, legkülső (leftmost-outermost) redukálható kifejezéseket (redexeket) választják ki. A gráfban ez a gyökértől balra haladva a legmagasabban fekvő kiemelt csomópont. A két stratégia között az a fő különbség, hogy míg a normal order egy nem applikált függvény törzsén belül is elvégzi a redukciós lépéseket, addig a call-by-name nem tesz ilyet, hanem megáll amint elér egy telítetlen absztrakciót (amelynek nincsen elég argumentuma ahhoz, hogy kiértékeljük). Ez utóbbi emiatt az úgynevezett gyenge fej-normálforma (weak head normal form) [3] állapotba hozza a kifejezéseket, ahol nincsen arra garancia, hogy az argumentumok között még nincsen kiértékelhető alkifejezés. A normal order esetén normálformába kerülnek a kifejezések, amit már nem lehet tovább egyszerűsíteni [13]. Ez a két stratégia a redukálást a 3. ábrán a $S(S(S)(S)(S(S)(K(K)(S))(S)))(S)(S)$ alkifejezéssel kezdené.
- Applikatív sorrend (applicative order és call-by-value): A legbaloldalibb, legbelső (leftmost-innermost) redexet redukálják először. Hasonló különbség létezik ezen két stratégia között is: a call-by-value nem végez kiértékelést az absztrakciók törzsén belül, míg az applicative order igen. A gyakorlatban ezek abban térnek el az előző pontban említett stratégiáktól, hogy egy függvény alkalmazása előtt – amennyiben annak minden paramétere rendelkezésre áll – az argumentumait értékeli ki, a fában lentől felfelé haladva. Ez a két stratégia a kiértékelést a 3. ábrán a $K(K)(S)$ alkifejezéssel kezdené. A legtöbb modern imperatív és szigorú nyelv (pl. C++, Rust, OCaml) ezt használja.

A stratégiák megválasztása azért kiemelten fontos, mert vannak olyan SKI-kalkulus kifejezések, amelyeknek a redukciója bizonyos stratégiákat alkalmazva végtelen ciklusba kerül. Tekintsük például az alábbi kifejezést:

$$KI((SII)(SII))$$

Applikatív sorrend alkalmazásával, a gép először megpróbálja az argumentumokat $((SII)(SII))$ teljesen kiértékelni, mielőtt a K kombinátort alkalmazná. Ez egy végtelen ciklust eredményez. Normál sorrend szerint viszont az algoritmus a legkülső redexet (a K applikációját) választja ki először, ezáltal a második argumentum (a divergáló kifejezés) eltűnik, és a kifejezés normál-formába kerül (I).

A kialakításban különösen figyeltem arra, hogy a redukciós logika moduláris maradjon, így az egyes stratégiák könnyen cserélhetők és összehasonlíthatók anélkül, hogy a motor alapvető működését módosítani kellene.



3. ábra. Egy kombinátor absztrakt szintaxis fája. A különböző kiértékelési módok kiválóan bemutathatóak rajta.

2.2.2. A λ redukciós gép

A λ -kalkulus kiértékelése bizonyos szempontból bonyolultabb, mint az SKI-kalkulusé. Ez részben abból adódik, hogy a λ -kalkulusban vannak változók. Létezik viszont a λ -kalkulusnak egy olyan alakja, amely – bár teljesen nem tud megszabadulni tőlük – egyszerűbbé teszi a kezelésüket. Ez az alak, a de Bruijn-forma³ [13, 1]. Ez a forma az úgynevezett de Bruijn-indexeken alapul: egy λ -kalkulus kifejezésben egy változó de Bruijn-indexe az a szám, amely azt jelzi, hogy az őt "kötő" λ -absztrakció hány szinttel van feljebb a kifejezés fájában. Például a $\lambda x.(\lambda y.x)$ kifejezésben az x változó de Bruijn-indexe 2, mivel az őt definiáló λ -absztrakció ($\lambda x.$) az két szinttel van feljebb. A kifejezés de Bruijn-alakja tehát $\lambda \lambda 2$.

Az én λ -kalkulus kiértékelőm ilyen alakban végzi el a kiértékelést, mivel így a kiértékelés során nem kell külön figyelni az úgynevezett α -ekvivalenciára. Ehhez meg kellett valósítanom a konverziót a tradicionális λ -kalkulus és a de Bruijn-forma között mindkét irányban. Ezek után a de Bruijn-alakra definiáltam a β -redukciót, amely megfelel egy λ -absztrakció kiértékelésének egy változóval. Ez után definiáltam a különböző stratégiákat, amelyek csak abban térnek el, hogy a kifejezésben éppen melyik alkifejezést β -redukálják.

Három stratégiát valósítottam meg: normal order, call-by-name illetve call-by-value [3]. Azért erre a háromra esett a választás, mert szerettem volna mind lazy és eager, mind normálformára és nem-normálformára hozó stratégiákat elérhetővé tenni.

2.3. A program tesztelése

A szoftver helyességének verifikálására a hagyományos egységtesztek mellett a Property-based testing (tulajdonság-alapú tesztelés) módszertanát alkalmaztam. Ennek során a teszt-keretrendszer automatikusan generál valid, véletlenszerű λ - és SKI-kifejezéseket, majd megvizsgálja a rendszer invariáns tulajdonságait.

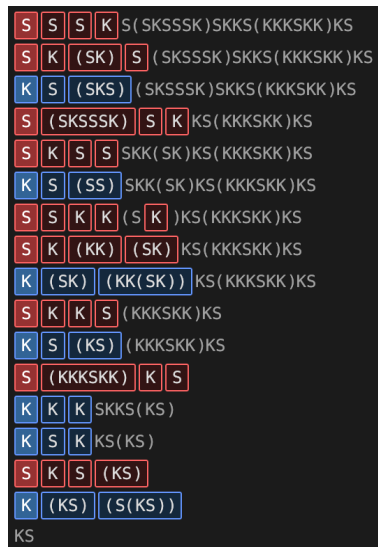
³Az általam használt változata a lokálisan névtelen de Bruijn-forma

A redukciós gépek kimenetének validálásához egy tesztelő orákulumot alakítottam ki [10]. Az orákulum egy már bizonyítottan helyesen működő, külső interpreter, amelyet egy elszigetelt Podman konténerben futtattam le a biztonság és a determinisztikus környezet garantálása érdekében. A tesztelési folyamat során a program és a Podman konténerben futó orákulum is kiértékelte ugyanazt a generált kifejezést, a teszt-keretrendszer pedig a két kimenet ekvivalenciáját vizsgálta black-box tesztelés (feketedoboz-tesztelés) formájában.

2.4. A vizuális redukciós elemek létrehozása

Egy oktatási eszköz esetében a belső folyamatok láthatósága kritikus jelentőségű, ezért a backendet felkészítettem arra, hogy úgynevezett *callback* függvényeket tudjon fogadni. Ezek a függvények egy olyan nézetet kapnak a belső gépről, amelyen olyan metódusok vannak definiálva, amelyekkel a teljes AST-t be lehet járni, szemantikai információt fűzve az egyes csúcsokhoz. Így az egyes redukciós lépések könnyen naplózhatóak, az alkifejezések különböző szerepe pedig elkülöníthető.

Algoritmusokat dolgoztam ki a redukálható kifejezések (redexek) detektálására és strukturált adatként (JSON) történő továbbítására a frontend felé. Ezek az adatok teszik lehetővé, hogy a webes felület vizuálisan kiemelje a soron következő redukciós lépést. Erre egy példa a 4. ábrán látható. A különböző kombinátoroknak más-más színük van, és meg vannak különböztetve a redukálható kifejezések az argumentumaiktól. A 2.2 fejezetben tárgyalt részfamegosztás módszere miatt egyes kombinátorok többször is ki vannak színezve. Ez arra utal, hogy a kifejezés fáájában az a látszólag két különböző kombinátor ugyanarra hivatkozik.



4. ábra. Egy redukciós lánc, ahol az egyes kifejezések szemantikailag vannak kiszínezve.

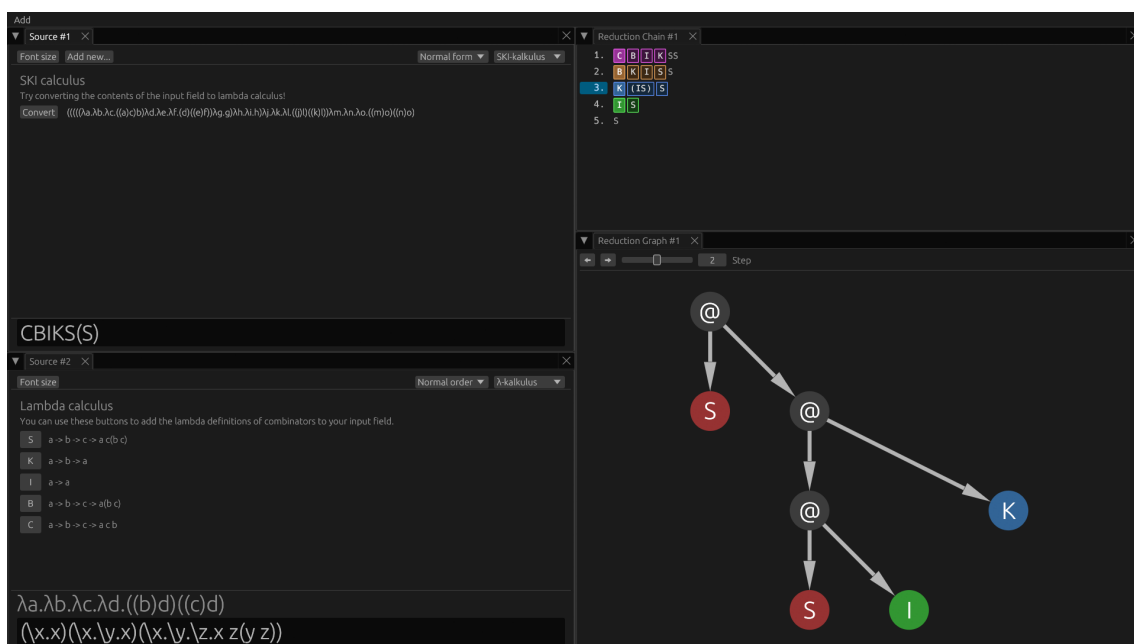
A redukciós láncon kívül a kifejezések gráfként való reprezentálását is implementálnom kellett. Az algoritmus az aktuális gráfállapot kinyeréséhez a G-machine memóriájában lévő gyökérelemből kiindulva egy mélységi bejárást (Depth-First Search) végez. A bejáró algoritmus külön kezeli a redukció során keletkező indirekciós csomópontokat: ezeket transzparens módon feloldja, és addig követi a mutatókat, amíg egy tényleges applikációt vagy kombinátort nem ér el. A végeredményt a G-machine egy lapos listává alakítja. Ebben a listában minden csomópont tartalmazza a típusát (App vagy Comb), a gyermekei azonosítóinak tömbjét, valamint a saját egyedi azonosítóját, amelyet közvetlenül az aréna belső mutatóiból származtatok. Ez a mutatómentes, lapos reprezentáció optimális a JSON-serializációhoz és a WebAssembly határon történő gyors átvitelhez, amely alapján a frontend keretrendszere képes felépíteni a vizuális gráfot.

2.5. A frontend fejlesztése

A frontend egy teljes mértékben kliensoldalon futó alkalmazás, amelyet Rust nyelvből WebAssembly-re (WASM) fordítottam.

Az Elm-architektúrára épülő reaktív felületen a felhasználó interaktívan gépelhet be kifejezéseket, kiválaszthatja a kívánt kalkulust és a kiértékelési stratégiát. A gördülékeny felhasználói élmény érdekében a háttérfolyamatokat Service Worker-ekbe (`worker.js`) szerveztem. Így a komplex és esetlegesen hosszú ideig futó redukciós fák kiértékelése nem fagyasztja le a felhasználói felületet, és a szoftver akár offline módban is képes teljeskörűen funkcionálni. A grafikus felületet az 5. ábrán mutatom be.

Az egui [7] grafikus keretrendszert használtam a felület kialakítására, amely egy úgynevezett *immediate mode* grafikus felület. Ennek a megközelítésnek az a különlegessége, hogy a felhasználói felület minden egyes képkockánál a nulláról épül fel és rajzolódik ki. A gombok, vagy beviteli mezők állapota nincsen eltárolva a memóriában két egymást követő frame között. Ez azt is jelenti, hogy a felület kirajzolása, és a felhasználói interakciók kezelése egyetlen lépésben történik.



5. ábra. A grafikus felülete a programnak.

2.6. Elérhetőség

A projekt elérhető GitHubon. A telepítési útmutató, futtatási utasítások és forráskód szabadon megtekinthetők [12].

2.7. Összefoglalás

A félév során sikeresen megvalósítottam a munkatervben kitűzött célokat. Áttanulmányoztam a releváns szakirodalmat, ami alapján megterveztem a Combinator Explorer program architektúráját. Kialakítottam a nyelvtanokat feldolgozó parsereket, implementáltam mind a λ -, mind az SKI-kalkulus G-machine alapú redukciós gépét, és mindezt egy WebAssembly alapú, interaktív frontenddel kötöttem össze. A Podman konténeres orákulumra épülő Property-based testing gondoskodik a motorok megbízhatóságáról. Az elkészült *Combinator Explorer* eszköz így sikeresen tölti be funkcióját: vizuális formát ad a funkcionális programozás matematikai alapjainak, hatékony és korszerű segédeszközt nyújtva a SKI- és λ -kalkulus kifejezések vizualizálására.

3. Irodalom, és csatlakozó dokumentumok jegyzéke

3.1. A tanulmányozott irodalom jegyzéke

- [1] Arthur Charguéraud: The Locally Nameless Representation. 49. évf. 3. sz., 363–408. p. ISSN 1573-0670. URL <https://doi.org/10.1007/s10817-011-9225-2>.
- [2] Keith D. Cooper–Linda Torczon: *Engineering a Compiler*. 2023, Elsevier. ISBN 978-0-12-815412-0.
- [3] Zoltán Csörnyei: *Lambda-kalkulus*. 2007. 05. 01., typotex. ISBN 978-963-9664-46-3.
- [4] H. B. Curry: Grundlagen der Kombinatorischen Logik. *American Journal of Mathematics*, 52. évf. (1930) 3. sz., 509–536. p. ISSN 0002-9327.
- [5] Evan Czaplicki: The Elm architecture. <https://guide.elm-lang.org/architecture/>, 2024. Utolsó megtekintés: 2026. május 19.
- [6] Firas Dib: Regex101: Build, test, and debug regex. <https://regex101.com>. Utolsó megtekintés: 2026. május 18.
- [7] Emil Ernerfeldt–contributors: egui: an easy-to-use immediate mode GUI in Rust. <https://github.com/emilk/egui>, 2026. Utolsó megtekintés: 2026. május 19.
- [8] Erich Gamma–Richard Helm–Ralph E. Johnson–John Vlissides: *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, Mass., 1995, Addison-Wesley. ISBN 978-0-201-63361-0.
- [9] Matt Godbolt: Compiler explorer. <https://godbolt.org>. Utolsó megtekintés: 2026. május 18.
- [10] W.E. Howden: Theoretical and Empirical Studies of Program Testing. *IEEE Transactions on Software Engineering*, SE-4. évf. (1978. július) 4. sz., 293–298. p. ISSN 1939-3520.
- [11] Thomas Mahler: Lambda is not a four letter word - Implementing a Functional Language with Graph Reduction. URL <https://thma.github.io/posts/2021-12-27-Implementing-a-functional-language-with-Graph-Reduction.html>. Utolsó megtekintés: 2026. május 18.
- [12] Orbán Levente: lexor. <https://github.com/levyry/lexor>. Utolsó megtekintés: 2026. május 18.
- [13] Benjamin C. Pierce: *Types and Programming Languages*. 1st. kiad. 2002. január, The MIT Press. ISBN 978-0-262-16209-8.
- [14] Chris Reade: *Elements of Functional Programming*. 1989, Addison-Wesley. ISBN 978-0-201-12915-1.
- [15] Adrian Sampson: Flattening ASTs (and Other Compiler Data Structures). URL <https://www.cs.cornell.edu/~asampson/blog/flattening.html>. Utolsó megtekintés: 2026. május 18.
- [16] M. Schönfinkel: Über die Bausteine der mathematischen Logik. *Mathematische Annalen*, 92. évf. (1924. szeptember) 3-4. sz., 305–316. p. ISSN 0025-5831, 1432-1807.

- [17] Henry Maurice Sheffer: A Set of Five Independent Postulates for Boolean Algebras, with Application to Logical Constants. *Transactions of the American Mathematical Society*, 14. évf. (1913) 4. sz., 481–488. p. ISSN 0002-9947.
- [18] Raymond M. Smullyan: *To Mock a Mockingbird: And Other Logic Puzzles Including an Amazing Adventure in Combinatory Logic*. 2000, Oxford University Press. ISBN 978-0-19-280142-5.
- [19] Stephen Wolfram: Combinators: A Centennial View. *Stephen Wolfram Writings*, 2020. december.
- [20] Stephen Wolfram: Combinators and the Story of Computation. *Stephen Wolfram Writings*, 2020. december.